Automatic differentiation algorithm in a simple convolutional neural network implementation

Kajetan Jeznach
Faculty of Electrical Engineering
Warsaw University of Technology
Warsaw, Poland
kajetan.jeznach.stud@pw.edu.pl

***Abstract*—Creating the own deep learning architectures is becoming increasingly uncomplicated these days with open-source libraries such as *Pytorch* and *Keras*. However, they often offer solutions as a black box, which is also not very fast and memory efficient. This paper presents a comparison of the automatic differentiation algorithms implemented in open-source libraries mentioned before and in a self-made library using Julia programming language. Aspects taken into consideration are speed, memory usage, flexibility and accuracy in a digit recognition task. As a result, an analysis has been created which led to the conclusion that Julia has a lot of potential in terms of fast and memory-efficient code.**

***Index Terms*—julia, convolutional neural network, automatic differentiation**

I. Introduction

Convolutional Neural Networks (hereafter: CNN) are the most popular [1] deep learning architecture used for image classification, object detection or semantic segmentation [2]. Its name derives from a mathematical operation called convolution, which is executed on an input image, and a kernel that extracts its features such as shape or contours [1]. In addition to the convolution operation, an important aspect that defines the performance of the network and its speed is the automatic differentiation algorithm [3], which was the aim of this study. Every single comparison made in this paper refers to the self-made library and *Pytorch* library.

II. State of the Art

Calculating the derivatives of the loss function relative to the weights in the CNN structure is a time-consuming and difficult task to implement from scratch. At the same time, information about these derivatives is crucial because it allows modifying the weights, so the value of the loss function is reduced. Among options there are: manual differentiation, which is not possible for millions of weights, symbolic differentiation where the expressions are getting very complicated due to the chain rule and numeric differentiation, which is subject to the truncation error. Therefore, it is convenient to use automatic differentiation algorithms. In *Pytorch*, the package responsible for that is *torch.autograd* [4], and in *TensorFlow – tf.autodiff* [5]. Because Python does not have built-in support for multidimensional arrays [6] these algorithms owe their speed to the implementation of core logic in C++ [3].

One of the types of automatic differentiation is forward mode. It is implemented using dual numbers [6]. The partial derivatives of intermediate variables are calculated in step with the original program. This process proceeds through the computational graph in the same order in which we evaluate the function. This requires as many passes as the number of function arguments. The other one is reverse mode, which uses an extension of the forward mode computational graph to enable the computation of a gradient by a reverse traversal of the graph. The intuition comes from the chain rule. It is a more efficient way to calculate gradients for scalar function of many variables [7], which is a classical deep learning setting. A programmer can select the type using the appropriate methods described in the documentation of *Pytorch* and *TensorFlow.*

The aim of the work was to create an automatic differentiation library and use it in CNN training to test its performance. One of the most popular database for doing this is a *MNIST* database [8]. It features a training set of 60000 examples, and a test set of 10000 examples of handwritten digits. It is easily available and anyone can use it to train its own neural networks. Moreover, in environments like *Pytorch* and *TensorFlow, MNIST* dataset can be downloaded using the built-in functionalities [9]. In this study, mentioned frameworks were used as a reference implementation for comparison of automatic differentiation algorithm. Another popular database is *ImageNet*. It contains around 1000 classes intended for using in image classification and object detection [1]. For self-made implementation, the chosen programming language was Julia. It owes its speed to a special combination of technologies that work very well with each other, such as expressive type system, aggressive code specialization against run-time types or carefully written libraries [10]. It was created in response to low efficiency of high-level languages [10], and its features are well suited for computational science [11].

III. Research methodology

Technologies used for creating reference and self-made model were: *Python 3.10.5* with *Pytorch 2.0.0* library and *Julia 1.8.5*. As an input dataset, MNIST database was chosen for recognizing digits. The processor being used during computation was *Intel Core i7-8750* with 6 cores and 12 threads. Its max frequency is 4.10 GHz. No graphics processor was used to ensure equal computation chances for both models.

For analysis, chosen criteria were: average time of learning

one epoch, accuracy achieved on test dataset, amount of memory used during one learning epoch. The architecture of the implemented neural network contains:

- convolution layer with 5x5 kernel, padding providing the same size of input as at the beginning,
- average pool layer with 2x2 kernel and stride 2,
- flatten layer, so there are 196 neurons,
- fully-connected layer with 196 neurons with sigmoid activation function,
- fully-connected layer with 100 neurons with sigmoid activation function,
- fully-connected layer with 10 neurons (one per digit),
- mean squared loss function.

IV. Julia's implementation description

Self-made implementation is based on an automatically generated computation graph, which task is to perform backward automatic differentiation on a specific graph created while user declares the architecture of the neural network. In the learning loop, the net forward and backward operations are performed. Thanks to the forward pass, all the values are known and then, with the backward pass, all the gradients are being computed. Those gradients are then updated, so the whole net improve its accuracy in digit recognition. Its speed was improved during creating the library by plenty of operations:

- preallocating the sizes of lists, matrices or vectors if the size is known before entering into loop,
- overriding the *backward* and *forward* functions for specific types of inputs performed during computing the graph such as mathematics operations and operations like convolution or average pooling,
- using Julia's macros such as *@views* and @. which optimizes the operations made on vectors and matrices,
- changing the style of performing a convolution operation.

An interesting solution was implemented to perform a convolution operation. In a primary version, the convolution was performed with a naive way, iterating through the every pixel of image with consecutive position of kernel on it. Although it leads to just iterating and performing several mathematic operations by iteration (so it is not that slow) it consumes a lot of memory and repeats some of the computations. There are two alternative ways to perform convolution with less memory usage. The first one is creating *im2col* function and replacing iterating through every pixel of image to multiplying kernel by an image, which will be a vector. The second one is converting kernel and image using Fast Fourier transform, performing matrices multiplying, and converting it back using Inverse Fast Fourier transform and absolute value operation made on output image. The second option was implemented to improve memory usage during computation. Code listing 1 presents the code to make *FFT* convolution. It uses a *FFTW* library built-in Julia programming language.

TABLE I
Comparison of FFT and Naive Conv

	FFT Conv	**Naive Conv**
Avg time $[\mu s]$	106.9	76.3
No of allocations	86	1153

Listing 1. Fast Fourier transform based convolution function

```
using FFTW
function fftconv(IMG, KER)
    temp_ker = zeros(size(IMG))
    temp_ker[1:size(KER,1),1:size(KER,2)].=KER
    I = fft(IMG)
    K = fft(temp_ker)
    J = ifft(I .* K)
    return abs.(J)
end
```

Replacing naive convolution function to Fast Fourier transform convolution was a huge improvement in memory usage. To prove that, *BenchmarkTools* package was used. The macro *@btime*, which calculates number of allocations and average time of execution, outputed the characteristics for both naive and *FFT* convolution. Results are shown in table I. Although there is a little regress in time, there is a huge improvement in amount of allocations.

Another comparison using macro *@btime* was performed to prove that all the actions mentioned before were accurate and truly made the library faster. Results are shown in table II. Like previously, there is a decent improvement in number of allocations. Average time of computing forward pass decreased more than twice, but the time of computing backward pass increased. Nevertheless, it is acceptable because the memory usage dropped.

TABLE II
Comparison of FFT and Naive Conv

		Primary implementation	**Improved implementation**
Forward pass	**Avg time** $[\mu s]$	657.6	307.9
	No. of allocations	5954	1376
Backward pass	**Avg time** $[\mu s]$	200.1	379.1
	No. of allocations	2067	1085

All mentioned actions led to reducing time of learning 30 epochs from 55 to 34 minutes. That gives a 1 minute and 8 seconds per epoch.

The accuracy of the trained neural network in Julia is 88.42%, which is a close result (not a surprise due to the same net architecture) to *Pytorch* model of 89.02%.

The package *BenchmarkTools* also provides a macro *@time* which does not perform hundreds of executions like *@btime*,

but just one. Using it on one epoch of learning, the result was: 69.8 seconds, 151.96 M allocations.

V. PYTORCH LIBRARY PERFORMANCE

As mentioned in section IV, the accuracy of the same architecture on test dataset was 89.02%. With package *tracemalloc* the memory usage was tracked during one epoch, and the results were: 152.7 M current allocations, 191.18 M peak allocations. The average time of learning one epoch was 2 minutes 40 seconds.

VI. SELF-CRITICISM ABOUT THE LIBRARY

It is worth it to notice that there were no changes made in graph structures declaration. This means that the library still has some potential to improve its speed by declaring specific types of variables in the graph declaration section. This action would lead to deleting the 'mutable' trait, which would significantly improve speed and memory usage.

VII. CONCLUSIONS

The amount of allocations during one epoch training is similar to each other (151.96M vs 152.7M). It can be concluded that the implemented computational algorithms are at a satisfactory level.

Time needed to train one epoch is much shorter for Julia's library. It is on average 1 minute and 8 seconds, while *Pytorch's* time needed to learn one epoch is about 2 minutes and 40 seconds.

The accuracy achieved on the testing dataset is almost identical for both implementations, but as mentioned before it was an expected result due to the same net architecture.

The figure 1 presents an example of wrong digit recognition (printed with *spy* function from *Gadfly* package). The identification of the digit is challenging for a human, and the implemented model also fails to accurately predict the correct label. Specifically, the assigned label for this particular test case was 2, while the model, including *PyTorch's* implementation, predicted it as 4.

As demonstrated above, it is not so hard to achieve faster and less memory-wasting solution than it is made in *Pytorch*. However, considering more complex architectures, *Python's* library would be definitely more user-friendly and will still provide decent memory usage and time. But one cannot forget that this framework was created over many years by experienced programmers. The presented implementation was a simple neural network with one convolutional layer (also just one kernel), one average pooling layer and 3 fully-connected layers. The tests should be repeated when it comes to create much more complicated architecture. But it can be assumed that the time put into upgrading the library would return.



Fig. 1. Wrong digit prediction

REFERENCES

[1] L. Alzubaidi, J. Zgang, A. J. Humaidi, A. Al-Dujaili, Ye Duan, O. Al-Shamma, J. Santamaria, M. A. Fadhel, M. Al-Amidie, L. Farhan, "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions", Journal of Big Data, vol. 8, 2021.

[2] J. Naranjo-Torres, M. Mora, R. Hernández-García, R. J. Barrientos, C. Fredes, A. Valenzuela, "A Review of Convolutional Neural Network Applied to Fruit Image Processing", Applied Sciences, vol. 10, 2020, art. number 3443.

[3] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, A. Lerer, Z. Lin, A. Desmaison, L. Antiga, "Automatic differentiation in PyTorch", 2017.

[4] Framework website: https://pytorch.org/

[5] Framework website: https://www.tensorflow.org/

[6] Bart van Merriënboer, O. Breuleux, A. Bergeron, P. Lamblin, "Automatic differentiation in ML: Where we are and where we should be going", CoRR, 2018.

[7] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind, "Automatic differentiation in machine learning: a survey", Journal of Marchine Learning Research, vol. 18, 2018.

[8] S. Kulkarni, S. Harnoorkar, "Comparative Analysis of CNN Architectures", International Research Journal of Engineering and Technology (IRJET), vol. 7, 2020.

[9] Xinghan Huang, Xiaofu Du, Hedan Liu, Wenkai Zang, "A Research on Face Recognition Open Source Development Framework Based on PyTorch", 2021 International Symposium on Computer Technology and Information Science (ISCTIS), 2021.

[10] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, "Julia: A Fresh Approach to Numerical Computing", SIAM Review, 2017.

[11] Vicky Singh, "Julia Programming Language – A True Python Alternative", Technotification, 23 August 2015.